LAST SIGNAL

Game design and production guide

A dark vehicle survival game built around satisfying driving, earned upgrades, personal refuges and a missing partner's trail.

UNREAL ENGINE 5.8 / PC FIRST / CONTROLLER READY

Three garages • Three vehicles • Twenty three implementation tasks

Prepared for Marius van der Westhuizen

29 October 2026 | Version 1.0 | Working title

Start with the first 4 km. Prove the driving and workshop loop before expanding the world.

Contents

Last Signal production pack 3

Game design 4

Technical design and contracts 7

Implementation roadmap 11

First three bases vehicles and upgrades 13

Art UI and asset bible 18

Claude and Codex setup and workflow 22

Testing performance and release gates 25

Production scope and monetisation 28

Narrative bible and spoilers 29

Sources assumptions and decision log 32

The companion ZIP contains the individual task cards, agent instructions, editable Markdown, balance seeds, asset inventory and copy-ready prompts. This book covers the design and the roadmap; use the smaller files during implementation.

Last Signal production pack

Version 1.0 | 29 October 2026 | Working title subject to name clearance

Build an original, single-player vehicle survival game: make one more desperate drive, bring home salvage, physically improve a battered car, and follow a missing partner's trail through increasingly unsettling safe houses. The target is responsive arcade driving with believable materials, weight and damage. This pack defines the game and the implementation contracts; it is not a finished Unreal project or a claim that the supplied designs have been playtested.

Decisions used to make this pack actionable

- Windows PC first; controller support from the first prototype; console ports after PC performance and funding gates. No mobile, multiplayer or VR in version 1.
- Third-person chase camera while driving; first-person exploration inside safe garages and small fenced forecourts. No on-foot combat or freely explorable open world.
- A compact campaign through three garages and three mechanically distinct vehicles, followed by an endless mode. A 4-6 hour first-playthrough target is a hypothesis, not a content promise.
- Fictional Greywater evacuation corridor: misty inland grassland, industrial outskirts and a reservoir upland. Local details can later be adapted to a South African setting without changing the systems.
- Premium base game; optional cosmetic packs after the core game is complete. The requested R1-R5 paints remain a pricing idea to validate, not a storefront commitment.
- Unreal Engine 5.8.x, with the exact installed patch and toolchain recorded in M00. Avoid engine upgrades in the middle of a milestone.

Read and use in this order

1. Read `docs/01_Game_Design.md` for the player experience and scope.
2. Follow `docs/06_Agent_Setup_and_Workflow.md` on the development PC.
3. Copy the agent instruction templates from `agent_templates/` to the Unreal repository root. Fill in verified local paths during M00.
4. Work through `tasks/M00_*.md` onward, in the order in `docs/03_Implementation_Roadmap.md`.
5. Give Claude only the current task, referenced documents and relevant code. Give Codex the relevant asset brief and style bible. Do not paste this whole pack into every session.
6. Tag the playable vertical slice at M15. Decide whether the driving is worth expanding before producing the remaining art.

Deliverables and authority

The PDF is a human-readable edition of this pack's overview and ten design documents. The ZIP adds individually scoped task cards, prompts, editable design sources, balance seeds, asset inventory and handoff templates. Markdown and JSON are the editable sources. Gameplay values in `data/balance_seed.json` are the tuning source of truth; the design documents explain their purpose. Update both when changing a rule.

Every generated asset still needs a provenance entry, import check and in-engine review. Image generation produces concepts, not automatically rigged vehicle meshes, clean UI layouts or production-ready MetaHumans. Neither Claude nor Codex has been connected to your local tools in this session.

Start with this prompt

Read CLAUDE.md, Docs/LastSignal/00_START_HERE.md, Docs/LastSignal/tasks/M00_Preflight.md and Docs/LastSignal/docs/06_Agent_Setup_and_Workflow.md. Execute M00 only. Inspect installed versions and the existing project before choosing commands. Create or validate the project scaffold, report tool availability, package the empty PC project if tooling permits, and update the handoff. Do not implement the game systems yet. Distinguish tested steps from blocked steps.

Game design

Player promise

Your car is your weapon, your workshop project and the last familiar object in a collapsing world. Short, dangerous drives fund visible improvements. Each secured refuge gives you a more personal home and another piece of your wife's journey. The excitement comes from recovering control under pressure; the sadness comes from what remains after the engine stops.

The inspirations describe functions: Zombie Highway supplies evasive driving and attached threats; Earn to Die supplies earned vehicle growth; Pacific Drive supplies attachment to a physical car; Dying Light supplies weathered atmosphere. Create original silhouettes, fiction, encounters, interface, audio and branding.

Design pillars

- Driving must feel good before art: readable hazards, responsive steering, weight without simulator friction, and recoverable mistakes.
- Upgrades change both handling and appearance. More armour trades agility and efficiency for impact protection; no vehicle makes the previous one obsolete.
- Garages provide calm, ownership and brief tactile work. Service is optional ritual after the tutorial, never a compulsory chore between every run.
- The road tells a controlled story inside a replayable structure. Mandatory clues and rewards never depend on lucky procedural rolls.
- Darkness sets the mood without concealing required information. Silhouette, sound and lighting must make hazards readable at speed.

Core and long-term loops

At a refuge, choose a destination and vehicle, review fuel and installed parts, then leave. Drive for roughly 6-10 minutes including hazards and deliberate slowdowns; collect salvage, fuel and evidence while choosing safer or richer branches. Reach a safe checkpoint to bank everything and refill, or fail and return with a small salvage recovery. Back at a garage, craft or find an upgrade, fit it, decorate one corner and inspect a new clue. Repeat until the next major refuge is secured.

The campaign has fixed regional milestones at route km 0, 12 and 30, with a finale at km 48. Intermediate checkpoints divide long distances into achievable legs. These are logical route distances, not a promise to store 48 km of unique terrain. Replayed legs have new encounter variants but retain the same authored landmark sequence. A player cannot unlock story progress by repeatedly driving around a loop.

Modes and failure rules

Campaign: reaching a forward checkpoint permanently unlocks it as a launch location. Outposts bank salvage, refill fuel and allow recovery, but full upgrades and decoration happen in garages. A map travel option between already secured refuges avoids forcing players to replay earlier roads for routine workshop access. Travelling home safely is a menu transition; unbanked cargo must first be banked at a safe location.

Endless: unlocked by completing the campaign, uses the same vehicle builds with a growing distance score and bounded difficulty. No critical story unlocks or exclusive power rewards depend on endless play. Record seed, rules version, assists and distance with each result; no online leaderboard in version 1.

Failure: banked inventory and ownership are safe; keep floor(30% of unbanked scrap), lose unbanked common crafting materials, and retain collected story evidence and first-time blueprint unlocks. Return to the selected secured refuge with the car recovered and serviceable. Basic repairs and a standard departure tank are free. Do not charge a repair debt that prevents another attempt. Explain recovery as assistance from an off-screen relay network, not resurrection.

Quitting is separate from failure. Normal exit creates a resumable run snapshot. Story evidence is journalled immediately. A corrupt or incompatible run snapshot falls back to the last committed checkpoint with an explanation and no banked-item loss.

Driving and impact combat

Use continuous steering, acceleration, braking and reverse. Keep roads broadly three lanes wide, with shoulders and occasional junctions; do not lock the car to discrete lanes. Add adjustable steering assist, traction aid, restrained anti-roll support and automatic transmission. Start with a high chase camera and limited player orbit; cockpit driving is deferred.

The default combat tools are steering, impact, boost and scraping attached zombies against marked obstacles. No manual gun aiming, weapons inventory or ammunition economy in the first release. Optional late development roof devices must pass a separate scope gate.

Zombies enter from verges or visible road spaces. A clinger can attach at a named vehicle socket, increase handling drag, and damage that local component. A side scrape dislodges clingers on that side; an intentional scrape causes a small, predictable body cost. At most four attached threats exist on one car. Do not make each attached zombie a fully simulated ragdoll.

An enemy or obstacle hit resolves once per impact window: relative speed, impact angle, vehicle mass class, ram rating and target type determine the outcome. Avoid real-world lethal speed thresholds. Low-speed bumps do little; clean frontal hits reward committed driving; hitting an immovable barrier is costly. Hit audio, suspension compression, a brief camera response and directional feedback sell the impact. Offer reduced gore and reduced camera motion.

Threat catalogue and encounter rhythm

Threat	Readable warning	Player answer	Initial scope
Drifter	Upright silhouette and slow crossing	Steer or ram	Slice
Clinger	Flank sprint and side warning	Scrape that side or avoid attachment	Slice
Brute	Broad stance, distinctive heavy sound	Avoid or use a strong ram at a cost	Region 2
Wreck choke	Reflective markers, visible gap	Brake and pick a lane	Slice
Fuel detour	Lit can symbol before junction	Trade time and exposure for range	Slice
Flooded rut	Water reflection and road warning	Slow down or use suitable tyres	Region 3

Alternate 20-45 seconds of active pressure with 15-30 seconds of recovery, adjusted by playtests. A director can lower spawn density after repeated failures but cannot fabricate an obstacle in the player's stopping distance. Reserve at least one feasible vehicle-width corridor. No unavoidable damage introduced by generation.

Economy and upgrades

Use one spendable currency, Scrap, plus three non-premium materials: Rubber, Alloy and Electronics. Blueprints unlock recipes and are permanent. Found complete parts are persistent unique inventory instances only after banking; quest-granted parts are granted transactionally at checkpoints. There is no energy timer, paid fuel or purchasable stat advantage.

Seed costs are 120 scrap for a first useful part, roughly 250-700 for mid-tier parts and 900-1500 for specialisations. Target a meaningful choice every 1-2 early runs and 2-3 later runs. The first tutorial provides materials for one wheel replacement. Measure actual earnings before balancing a grind around these numbers.

Each vehicle has engine, fuel, suspension, ram, utility and four wheel slots. Cosmetic paint, decals, mirrors, roof trim and cabin ornaments use separate non-stat slots. Weight changes only come from gameplay parts. Four individual tyres use one compatible wheel family per car; mixed tyres are allowed only when radius matches. UI warns about mixed grip, while mismatched radii block departure.

Crafting creates an item; installation changes the vehicle. Replaced parts return to inventory. Sell or salvage parts only with an explicit confirmation, and never destroy the only compatible wheel needed to leave. Unlocks do not auto-equip parts or reset a favourite build.

Workshop and home customisation

Walk to the bench, vehicle, wardrobe, radio or route board. The same functions are reachable through an accessible workshop menu. Repair and upgrade interactions share one focus target and one confirm action, with clear invalid-state text.

Wheel sequence: park in the bay; engage service mode; raise the axle with the jack; loosen an abstracted fastener group; remove the old wheel; carry and align the new wheel to a snap target;

tighten; lower. No per-bolt physics or precision hand IK is required for the slice. Cancel returns safely to the last valid state. After doing it once, Quick Fit performs the same validated transaction without the animation. Both methods have identical stats and costs.

Home decoration uses a placement mode with snap rotation, grid/free positioning, wall/floor categories and undo. Start with 20 approved objects across benches, storage, seating, rugs, lights and personal keepsakes. Block exits, vehicle clearance and required station exclusion volumes from placement. Decorations carry no performance buffs; unlocked stations provide functionality. Save each base layout independently and offer Reset Layout without deleting owned items.

Narrative interaction

Mandatory clues sit on the route board, checkpoint arrival scene or another guaranteed position. Optional personal objects add context. The journal records exact observations separately from the protagonist's interpretation. Garages have short, optional environmental stories; no mandatory ten-minute dialogue scene interrupts a driving session.

The full spoiler treatment is in document 09. The ending is authored and resolves the wife's fate; procedural ambiguity must not substitute for a coherent conclusion.

Scope boundaries and success gates

Ship three cars, three garages, three region kits, three enemy behaviours and one complete story. Exclude multiplayer, survival hunger/thirst, open-world foot travel, structural base construction, dynamic vehicle crumple simulation, fully simulated loose cargo and live-service seasons.

Prototype gate: five independent testers understand steering and identify why they failed. Slice gate: at least four of five voluntarily choose another run, and all can replace a wheel and launch using only a controller. These are small-sample development signals, not market forecasts. Cut features before compromising feel, stability or readability.

Technical design and contracts

Runtime shape

Use a C++ core with Blueprint children for assembly and tuning. Keep transactional inventory, persistence, procedural scheduling and rule calculations in testable C++; use Blueprint for presentation, interactions and art hooks. Python is an editor/asset-pipeline tool, not packaged gameplay [S05]. Start with one runtime module `LastSignal` and one optional editor module `LastSignalEditor`; do not build a framework of many plugins before the slice.

Use separate garage maps and one streamed road runtime. Loading a destination garage is an intentional safe transition. World Partition can help an authored region when justified, but is not the generator for an endless road. The first implementation uses a bounded chunk manager with authored connectors, reusable actors and data-driven content.

Responsibilities and stable boundaries

Owner	Responsibility	Must not own
ULSProfileSubsystem	Profile, banked inventory, unlock ledger, save commits	Live physics
ULSRunSubsystem	Run lifecycle, seed, unbanked cargo, checkpoint and failure resolution	UI widgets
ALSRouteDirector	Chunk readiness, safe corridors, encounter schedule	Economy mutation
ALSVehiclePawn	Driving, component damage, sockets, presentation events	Permanent unlocks
ULSVehicleBuildComponent	Installed part IDs and evaluated stats	Crafting wallet
ULSWorkshopService	Craft/install/remove validation and atomic changes	Input polling
ALSGarageCharacter	Walk, carry visual and interaction trace	Inventory authority
ULSUIRouter	Common UI screens and focus restoration	Save-file writes

Names are proposed project types, not existing engine APIs. Confirm engine APIs against the installed headers before implementing. Gameplay Ability System and Mass are unnecessary for the first bounded enemy counts; add them only after profiling or requirements justify the migration.

Data definitions

Definitions are Primary Data Assets with stable string-like IDs and soft asset references. Instances store IDs and state, not transient UObject pointers. Blueprint display names can change without changing save identity. Keep definitions versioned and validate references during cook.

- VehicleDefinition: ID, chassis asset, compatible part tags, base mass, fuel/range tuning, socket map, steering/torque profiles and default build.
- PartDefinition: ID, slot category, compatibility tags, recipe, stat modifiers, visual asset, unlock condition and preview text.
- PartInstance: unique instance ID, definition ID, condition and cosmetic overrides. An instance exists either in inventory or exactly one installed slot.
- RoadChunkDefinition: ID, length, entry/exit transforms, lane widths, speed band, biome tags, clear zones, optional sockets and collision/visual assets.
- EncounterDefinition: stable ID, tags, cost, warning distance, spawn bounds, cooldown and mutually exclusive hazards.
- CheckpointDefinition: ID, route distance, arrival map or outpost, reward IDs and mandatory clue IDs.
- DecorationInstance: unique ID, definition ID, base ID, local transform, surface ID and colour selection.
- ClueDefinition: ID, exact text/audio, prerequisite, guaranteed placement, journal order and spoiler tier.

Start with the included JSON for tuning and CSV for art planning. They are seed specifications, not files Unreal automatically imports into these structures; M01 builds and validates the import or equivalent Data Assets.

Events and commands

Use commands for mutations and events after success. `TryInstallPart(vehicleId, slotId, partInstanceId, expectedRevision)` returns success or a structured failure; UI never edits arrays directly. Publish `BuildChanged`, `CargoChanged`, `CheckpointCommitted`, `ClueRecorded`, `RunEnded` and `InteractionStateChanged` with IDs and compact payloads. Bind listeners once and remove bindings on teardown.

Run states: AtRefuge, Preparing, Driving, Arriving, Failed, Resolving, Suspended. Only one state transition owns reward settlement. A checkpoint commit uses a unique transaction ID, banks cargo, grants once-only rewards, advances the checkpoint and writes the durable profile before presenting success. Retrying that transaction must not grant twice.

Distance and procedural road

Maintain three different measures: `RunDistanceM` for forward progression in the current attempt; `CampaignFrontierM` for the furthest committed story checkpoint; and `LifetimeOdometerM` for statistics. Reverse movement and circles do not increase route progress. Route position comes from ordered chunk index and monotonic spline progress, rather than accumulated absolute wheel travel. Branches rejoin one logical checkpoint distance and cannot skip required milestones.

Begin with 250 m nominal chunks, five ahead and two behind the current chunk. Treat these as starting values; actual chunk length and visibility determine the active count. At 110 km/h, five forward chunks give about 41 seconds of look-ahead. Schedule data preparation first, then asset loading, collision readiness, gameplay placement and decorative PCG work. All lane collision and mandatory gameplay must be ready before entry.

The generation seed is a stable hash of campaign seed, route segment ID, chunk index and generator version. Keep separate random streams for layout, encounter and decoration. Log resolved chunk/encounter IDs, so a failing seed can be replayed even after cosmetic changes. Deterministic layout does not imply cross-platform deterministic Chaos physics.

PCG dresses the corridor with vegetation, rocks, poles and ruins; reserved volumes protect lanes, sightlines, garage doors, rewards and required story scenes. Prefer pre-generated scenery variants in the slice. Introduce runtime PCG only after M07 is stable, with generation/cleanup radii and work budgets measured in a packaged build [S03]. Critical route selection is owned by game code, not a free-form PCG graph.

Keep one fixed global weather/time profile per run initially. Change weather at checkpoint transitions. This avoids spending the art budget on every lighting condition before the route works. Stop repetition with a recent-chunk exclusion list, biome grammar, direction changes within connectors and authored landmarks. Seed validation checks every branch for sufficient clearance and reachable exits.

Long-distance coordinates and streaming recovery

Use an int64 logical route index/distance alongside local transforms. Validate an origin-rebase implementation at a controlled streaming boundary, transforming the active road, vehicle, physics state, attachments and VFX together; do not assume large-world coordinates alone solve an unbounded journey. Test after a simulated 1,000 km. Save logical route coordinates, not only a huge world-space vector.

If a required chunk is not ready, extend a preloaded safe straight segment and defer the next encounter. A checkpoint loading transition is the last fallback. Never expose missing collision, hide a hard stop inside fog, or teleport the car without a visible transition. Chunk pooling must reset timers, attachment lists, collision, delegates, audio and loot state.

Vehicles and damage

Prototype with Chaos Vehicles [S06] and a known-good four-wheel rig. Keep wheel bones and visible interchangeable wheel meshes aligned through a tested import convention. Wheel upgrades may modify grip and visual tread without changing rolling radius in version 1; reconfigure runtime wheel parameters only through verified APIs. Rebuild the parked vehicle physics state when needed.

Apply assists through bounded, delta-time-aware control. Test identical scenarios at 30, 60 and 120 fps; do not assume presentation tick timing equals physics timing. Use query-only sensors for clinger sockets, one target resolution window per collision and simple convex collision for dynamic parts. Cosmetic break-off panels are pooled and capped; damage swaps/decals replace soft-body crumpling.

Vehicle modules: engine, body, wheels and fuel. Losing engine/body integrity ends the run; a damaged wheel degrades handling but does not become unrecoverably random. Fuel exhaustion permits a brief coast, then a clear recovery action. Upgrades modify baseline stats through one evaluator: additive values first, multiplicative values second, then clamps. Recompute from defaults to avoid stacking a bonus again after loading.

Transactional workshop

Keep a logical service transaction separate from carried visual props. Validate vehicle parked, service mode, compatible slot, ownership, recipe or inventory state, and current revision before changing anything. Reserve the incoming part; never deduct both at craft time and again at installation. During a wheel animation the old part is visually detached, but logical replacement commits only on successful completion. Cancellation returns visuals and reservations to their original state.

Save only settled workshop state. If interrupted mid-animation, reload the last committed build with both inventories intact. Repeated confirm inputs are debounced by transaction ID. Quick Fit calls the same service command and bypasses only presentation.

Persistence and migration

Profile schema: schemaVersion, contentVersion, profileId, revision, banked resources, owned parts, vehicle builds, checkpoints, once-only reward ledger, clues, per-base decoration layouts and settings. Run snapshot: seed/version, route location, vehicle state, cargo, consumed pickups and resolved encounter IDs. Keep purchase entitlement data separate from editable gameplay saves.

Implement A/B save slots with generation numbers and integrity checks. Write the inactive slot, read-validate it, then select the newest valid generation at load. Preserve the older valid slot; a

checksum detects corruption but is not an anti-cheat system. Test interrupted writes, disk-full errors, migration of the previous schema and missing/retired asset IDs. Unknown cosmetic IDs fall back; unknown core part IDs restore a compatible stock item with a logged migration.

Input and UI

Enhanced Input supplies mutually exclusive driving, on-foot, service and placement contexts [S09]. Common UI supplies layered screens, controller navigation and glyph routing [S10]. Pause/menu input consumes gameplay actions. Centralise device switching so mouse jitter does not flicker controller prompts. A controller disconnect pauses single-player driving and shows reconnect guidance.

Content folders

Use `Source/LastSignal/{Core,Vehicle,Route,Workshop,Garage,Narrative,UI}`, `Content/LastSignal/{Core,Vehicles,World,Garages,Characters,UI,Audio,Data}`, `Scripts/{Unreal,Blender}`, `Tests/`, and `Docs/LastSignal/`. Put external assets under `Content/External/<Vendor>` and preserve licence information. Runtime code must not import editor-only modules. Use soft references and explicit cook rules for dynamically selected assets.

Implementation roadmap

Build in dependency order. Each task file contains a bounded assignment, outputs, implementation sequence, evidence and stop conditions. The owner is accountable for integration; another agent may review without changing the same files. Expect to split a task into smaller sessions when its acceptance checks cannot fit into one focused implementation.

Task	System	Depends on	Owner
M00	Toolchain and packaged blank project	None	Claude
M01	Definitions and profile persistence	M00	Claude
M02	Input and mode switching	M01	Claude
M03	Vehicle feel test track	M02	Claude
M04	Impact and component damage	M03	Claude
M05	Fuel cargo failure and checkpoint loop	M01 M04	Claude
M06	Garage walk and interaction	M02 M05	Claude
M07	Bounded road streaming	M03 M05	Claude
M08	Drifter clinger and encounter director	M04 M07	Claude
M09	Crafting and build transactions	M01 M06	Claude
M10	Manual wheel replacement	M09	Claude

Task	System	Depends on	Owner
M11	Route milestones and clue journal	M05 M07	Claude
M12	Controller UI and settings	M02 M09 M11	Claude
M13	Garage decoration	M06 M09 M12	Claude
M14	First-region production art and sound	M03 M07 M10 M12	Codex plus human review
M15	Packaged vertical slice gate	M08 M10 M11 M12 M13 M14	Claude integration
M16	Runtime scenery and long-drive resilience	M15	Claude
M17	Vehicles two and three	M09 M14 M15	Codex assets then Claude
M18	Garages and regions two and three	M11 M13 M16 M17	Codex assets then Claude
M19	Finale and endless mode	M16 M18	Claude
M20	Performance accessibility and save hardening	M19	Claude plus human testing
M21	Optional cosmetic entitlement integration	M20 and business gate	Claude
M22	PC release candidate and console feasibility	M20 M21 or explicit skip	Human owner plus Claude

The vertical slice

The slice is one polished 0-4 km leg, one walkable Mile Zero garage, the Kestrel wagon, two threats, one authored clue, one physical wheel upgrade and a small set of decorations. It includes arrival, banking, failure recovery, saving, relaunching and complete controller navigation. Use the 4 km outpost to demonstrate the loop without needing the second garage.

Use six to eight road chunks with variants and one lighting setup. A good slice can prove the experience in 15-25 minutes. It is not a miniature of every planned feature. No paid store, second car, elaborate cinematics, voiced dialogue system or full runtime PCG is required to judge it.

Sequencing rules

Finish M03 before polishing vehicle art; finish M05 before balancing prices; finish M07 before expanding scenery; finish M10 before authoring all workshop animations. Character hands and placeholder arms suffice until the interaction is proven. M14 art can be prepared in an isolated branch after contracts stabilise, then merged with one integrator.

At every task: inspect current state, write a short local plan, implement one coherent slice, compile, run the named checks, package when required, record actual results and commit. A missing external tool is a blocker to that check, not a pass. If two repair attempts fail for the same reason, reduce to a reproducible case rather than letting an agent redesign unrelated systems.

Work estimates

These are planning allowances for human-led development with AI assistance, not measured productivity promises. They include iteration and integration and should be recalibrated after M03 and M15.

Gate	Cumulative effort	Meaning
Playable greybox loop	160-320 hours	Feel and progression work with placeholders
Polished vertical slice	500-1000 hours	One presentable repeatable leg
Compact PC release	3000-6500 hours	Three regions, content, QA and launch preparation
Console ports	Separate estimate after access	Devkits, platform work, certification and testing

At 20 hours per week, 3000-6500 hours is approximately 150-325 working weeks before holidays and interruptions. AI can reduce scripting and repetitive setup time, but art direction, driving feel, testing and platform work remain substantial. To shorten the schedule, ship one polished region and fewer vehicles as an explicitly smaller product; do not assume generation eliminates production costs.

First three bases vehicles and upgrades

All names and dimensions are proposed design specifications. Build original forms without real vehicle badges or copied game landmarks. Dimensions use metres; Unreal uses centimetres after verified import. Vehicle figures are gameplay tuning seeds, not claims about real vehicles. Major vehicle rewards are free and permanent on first checkpoint completion.

Progression map

Route km	Refuge or reward	Guarantee
0	G01 Mile Zero Service and V01 Kestrel	Starter car, jack, first wheel kit
4	Weighbridge outpost	Bank, refuel, first route clue
8	Pump station outpost	Free tyre kit and personal object
12	G02 Redline Depot and V02 Mule	Garage, stock pickup, alloy recipes
18	Water tower outpost	Free suspension kit and clue
24	Quarry shelter outpost	Free utility module and evidence
30	G03 Stillwater House and V03 Warden	Garage, stock recovery vehicle, final region
36	Dam service outpost	Free specialist tyre kit and clue
42	Relay shelter outpost	Final lead and preparation space
48	North Relay finale	Story resolution and endless mode

Every reward records a stable grant ID. Returning to a checkpoint never duplicates a free car or rare part. Ordinary replay salvage remains available.

G01 Mile Zero Service

Purpose: a small, credible shelter that teaches working on a vehicle and makes the first departure emotionally clear. A shuttered roadside repair shop has been barricaded by its last owner. Cold daylight falls through dirty glass; one amber work lamp lights the wheel bay. Mara's marked road map lies near a chipped blue enamel cup.

Blockout: workshop 14 x 10 m with a 5 m clear height, rear living room 6 x 5 m, fenced forecourt 18 x 16 m. Single 4 x 3.5 m vehicle door; 1.2 m clear walking paths and a protected service clearance around the parked car. The 1.8 m player reference must reach the bench and wheel without crouching.

Stations: jack and wheel rack beside the bay; scrap bench at the rear wall; route board beside the exit; radio and bed in the living room; paint rack unlocked by the tutorial. Decoration zones: living-room floor, two walls, shelves and a small forecourt strip. Do not place furniture in the service bay.

Asset set: corrugated roof, plaster/brick wall modules, shutter, office partition, two windows, workbench, hydraulic jack, tyre rack, route board, radio, cot, cup, fuel can, toolbox, fluorescent fixture and fence kit. Reuse the tools in later garages. Story focal point is the map and cup, never a large exposition wall.

Concept prompt: original game environment, abandoned inland roadside service shop, modest corrugated roof and worn plaster brick, a single believable vehicle bay, oil stains and restrained repair clutter, cold overcast daylight with one warm task lamp, quiet refuge inside a dark zombie evacuation world, realistic PBR surfaces, desaturated blue-grey and rust, readable walkways, no people, no logos, no text, 16:9 eye-level interior. Produce a separate top-down layout study using the stated

dimensions; technical dimensions remain authoritative if the generated image disagrees.

V01 Kestrel estate wagon

Role: light, economical, nimble starter. Long roof, narrow upright headlights, slightly squared wheel arches, repaired rear quarter and a blank grille make a recognisable silhouette without copying a manufacturer. A faded off-white body has one slate-blue replacement door. Interior: cloth seats, worn analogue gauges, a taped photograph recess and an empty radio cradle.

Dimensions: 4.45 x 1.75 x 1.48 m; 2.62 m wheelbase; 0.31 m wheel radius. Four wheels; modest ground clearance. Seed mass 1280 kg, top speed 110 km/h, fuel capacity 18 game-units, nominal range 4.5 km, cargo 8 slots, body integrity 100. Fuel is deliberately compressed for the arcade loop and is not litres.

Part ID	Upgrade and visible change	Gameplay seed	Scrap
P_K_TYRE_T1	Retread road wheel, thicker worn tread	+8% dry grip; per wheel	120
P_K_ENGINE_T1	Refurbished intake, subtle bonnet vent	+12% engine force	260
P_K_FUEL_T1	Secured auxiliary tank under rear floor	+25% fuel capacity	220
P_K_RAM_T1	Narrow welded bumper with tow eyes	+20 ram rating; +45 kg	200
P_K_SUSP_T1	Fresh dampers and short lift	+10% stability tuning; +10 kg	280
P_K_UTIL_T1	Bolted rear salvage rack	+4 cargo slots; +20 kg	240

Upgrade silhouette progression: intact road car; visible tyres and useful bumper; patched touring survivor with a compact rack. Preserve glass, lights and recognisable road-car proportions. No enormous blades or weapons in the starter set.

Concept prompt: original fictional late twentieth-century compact estate wagon, long practical roof, restrained square proportions, faded off-white paint, one slate-blue replacement door, small welded bumper and believable all-season tyres, gritty but mechanically functional, no brand resemblance or logos, no text, neutral studio light, full vehicle visible, three-quarter front view. Generate separate left, right, front, rear and top references from the approved design; retain wheelbase, wheel count and window shapes. Do not render every upgrade at once.

G02 Redline Depot

Purpose: the first feeling of strength and ownership, with enough space to compare two vehicles. A deserted municipal transport workshop sits beside railway sidings and a disused loading platform. Red oxide beams and dirty skylights frame a sunken inspection pit. A small upstairs dispatch room overlooks the bay.

Blockout: main hall 24 x 16 m, clear height 7 m; dispatch room 8 x 5 m; fenced yard 30 x 22 m. One active vehicle pad and one static display pad keep physics and UI simple. Stairs are 1.4 m wide. Guard the inspection pit with a removable service cover and safe navigation collision.

Stations: powered bench, parts shelves, paint corner with ventilation fan, route board in dispatch, bed alcove and salvage locker. Functional unlocks are higher recipe tiers and vehicle selection, not damage buffs from furniture. Decoration zones: dispatch, lounge, marked walls and courtyard pockets.

Asset set: industrial truss, brick infill, skylight, pit module, stairs/rail, roller door, depot gantry as static scenery, lockers, oil drums, tool trolley and dispatch furniture. Build at least 70% of walls/props from common kits. Distinct story props: Mara's luggage tag and a dispatch ledger with two contradictory dates, rendered as authored readable UI when inspected.

Concept prompt: original abandoned transport depot converted into a survivor workshop, red oxide roof beams, worn masonry, dirty skylight, single active car bay with covered inspection pit, raised dispatch office, organised salvaged tools, restrained amber work lighting in cold industrial dusk, dark quiet mood, plausible architecture, clear circulation, no logos or generated lettering, 16:9 wide interior.

V02 Mule utility pickup

Role: balanced salvage carrier. Broad cab, upright windscreen, open bed, utilitarian grille and a short straight bonnet. Visual DNA shares the starter's practical era but not its body panels. Dusty olive paint, grey primer repairs and a folded canvas bed cover. Avoid military styling.

Dimensions: 5.05 x 1.90 x 1.85 m; 3.02 m wheelbase; 0.37 m wheel radius. Seed mass 1880 kg, top speed 100 km/h, fuel capacity 24 units, nominal range 6.0 km, cargo 12 slots, body integrity 150. Stronger ramming and cargo capacity trade against acceleration and tight avoidance. Stock fuel range is a baseline; guaranteed cans support 6 km legs with margin.

Part ID	Upgrade and visible change	Gameplay seed	Scrap
P_M_TYRE_T1	All-terrain wheel with chunky shoulders	+18% wet grip; per wheel	280
P_M_ENGINE_T1	Rebuilt torque system and intake box	+15% engine force	520
P_M_FUEL_T1	Protected long-range tank	+30% fuel capacity	480
P_M_RAM_T1	Angled beam bumper and side braces	+35 ram rating; +100 kg	600
P_M_SUSP_T1	Heavy springs and fresh shocks	+15% stability tuning; +25 kg	520
P_M_UTIL_T1	Utility cradle with fuel scavenger	+20% fuel pickup yield; +35 kg	650

Concept prompt: original rugged civilian utility pickup, practical upright cab and open cargo bed, dusty olive body with dull grey repairs, mechanical plausibility, four all-terrain wheels, slightly widened stance, believable welded front beam and cargo tie-downs, dark apocalypse road game, no real brand features, no text, neutral studio background, full vehicle three-quarter view. Supply stock and tier-one attachment views as separate images.

G03 Stillwater House

Purpose: a refuge that feels almost like a home, making the final journey harder to leave. A weatherboard/concrete reservoir caretaker house stands above a drained inlet. Its attached boathouse has become a workshop. The rooms feel carefully maintained, although no one answers the radio.

Blockout: house 12 x 10 m, garage 16 x 12 m with 6 m clearance, covered passage 4 x 6 m and fenced lookout deck 8 x 5 m. No accessible lake, boat simulation or large explorable estate. Workshop clearance accommodates the Warden. Use horizon scenery to imply a larger place.

Stations: universal parts bench, paint wall, route desk facing the reservoir, bedroom, kitchen and radio nook. Decoration focus: books, lamps, rugs, collected objects and a small memorial shelf. The initial composition supports both comfort and unease. Daylight stays cold; personal lamps warm the player-built interior.

Asset set: house wall/door/window kit, concrete plinth, pitched roof, timber deck, workshop doors, simple kitchen, sofa, desk, curtains, lamps, railings and reservoir vista cards/meshes. Hero story props: repaired coat, taped radio cassette and an identifiable scratch on the desk. Furniture reuses material families from G01/G02.

Concept prompt: original secluded reservoir caretaker house with attached high workshop, weatherboard above concrete foundations, mist over a receding lake, practical lived-in rooms with warm task lamps against cold blue-grey daylight, somber refuge, restrained repairs and human traces, no fantasy castle, no luxury mansion, no people or text, cinematic yet playable architectural proportions. Generate garage interior and home interior separately.

V03 Warden recovery truck

Role: four-wheel, two-axle compact recovery truck, a deliberate heavy bruiser rather than an unstoppable final upgrade. Short cab-over front, broad steel recovery bed and a folded nonfunctional boom as silhouette detail. No six-wheel rig, articulated trailer or working crane in version 1. Faded ochre body with charcoal protective panels.

Dimensions: 5.80 x 2.05 x 2.35 m; 3.35 m wheelbase; 0.42 m wheel radius. Seed mass 2850 kg, top speed 90 km/h, fuel capacity 30 units, nominal range 6.5 km, cargo 16 slots, body integrity 210. Wide turning radius and slower recovery from steering errors keep the lighter cars useful.

Part ID	Upgrade and visible change	Gameplay seed	Scrap
P_W_TYRE_T1	Heavy reinforced wheel	+20% tyre durability; per wheel	450
P_W_ENGINE_T1	Heavy cooling and torque package	+18% engine force	1050
P_W_FUEL_T1	Protected saddle tank pair	+25% fuel capacity	950
P_W_RAM_T1	Braced wedge bumper with tow eyes	+50 ram rating; +180 kg	1200
P_W_SUSP_T1	Load-rated dampers	+18% stability tuning; +45 kg	900
P_W_UTIL_T1	Recovery storage cage	+6 cargo slots; +65 kg	1100

Concept prompt: original compact civilian cab-over recovery truck, exactly four wheels on two axles, folded static recovery boom, broad practical steel bed, faded ochre paint with charcoal repair panels, believable reinforced bumper, heavy but road legal proportions, gritty post-collapse setting, no real manufacturer cues, no military turret, no text or logos, neutral studio three-quarter full-vehicle view. Keep boom folded in all gameplay references.

Common optional specialisations

All cars can unlock a 700-scrap Economy Tune (+12% nominal range, -8% engine force), an 800-scrap Reinforced Shell (+20% body integrity, +8% mass) and a 900-scrap Quick Steering package (+10% steering response with reduced high-speed assistance). These are alternative compatible parts in engine, ram and suspension slots respectively, not extra stacking slots. The tuning evaluator must include each tradeoff in preview text. Prices and effects require playtesting.

Asset acceptance for all six briefs

Approve one stock silhouette and one colour/material sheet before producing variants. Require consistent dimensions, sockets, pivots, material slots and damage regions. Deliver a `.blend` source, documented Unreal export, texture set, collision, preview turntable and an import validation result. Image references alone do not complete an asset task. The initial production inventory is in `data/asset_manifest.csv`.

Art UI and asset bible

Visual language

Believable materials and grounded proportions carry the fidelity; bold silhouettes and restrained exaggeration carry the arcade readability. The world is damaged by weather, repair and abandonment, not uniformly covered in noise. Keep 60% of each important surface relatively quiet, with grime concentrated at seams, runoff paths, wheel arches and handling points.

Palette: charcoal #181B1D, cold slate #53616A, bone #E5DFD1, rust #965640, work amber #D8A247, faded olive #666C54, danger red #D75B50. Red marks immediate danger, amber marks actionable service states, bone carries text, and a muted blue-green #7DB7AE marks confirmed success. Do not use colour as the only state indicator.

Exterior light is soft overcast, late dusk or thin mist; the first slice uses overcast dusk only. Garages contrast cool ambient light with warm practical lamps. Avoid crushed black driving lanes, excessive bloom, constant wetness, neon RGB gear, screen-wide grunge overlays and comic-outline zombies. Lights, glass and rubber must remain distinct from painted metal under every quality preset.

Asset families and modularity

Build three reusable environment families: service masonry and corrugated metal; depot brick and red oxide steel; caretaker timber and concrete. Common props share metal, rubber, cloth and painted wood master materials. Use trim sheets and tiling surfaces for large areas, decals for local storytelling, and unique textures only where the camera benefits.

Kit grid is 0.5 m with major 2 m and 4 m wall pieces. Floor modules are 4 x 4 m; connectors must align without visible light or collision seams. Vehicle geometry has separate doors/panels only where animation or damage needs them. Wheel origins sit at hub centres; attachment sockets use stable names such as Wheel_FL, Wheel_FR, Wheel_RL, Wheel_RR, Ram_Front, Utility_Rear and Ornament_Dash. Publish transforms in a manifest and never silently move them after an animation is authored.

Starting production budgets

Asset	Starting budget	Validation
Hero vehicle	80k-150k conventional LOD0 triangles	Profile close workshop and moving road views
Vehicle materials	6-8 slots including glass	Prefer shared atlases and masks
Vehicle textures	2K sets; selected 4K body if justified	Check actual residency and blur at distance
Common prop	1K or shared trim; simple collision	Judge silhouette at intended distance
Hero prop	2K where close inspection needs it	Compare with cheaper variant
Road zombie	15k-35k LOD0 with cheaper distant LODs	Budget animation and shadows, not only triangles
Architecture	Nanite candidate for static opaque detail	Inspect material cost, fallback and collision

These are project starting budgets, not engine limits or guaranteed optimal counts. Test Nanite on appropriate content; do not remove collision, material, animation or memory budgets because geometry streams efficiently [S07]. Keep the proven conventional vehicle rig until a measured Nanite skeletal experiment wins on the target hardware.

Image generation brief contract

Every request names asset ID, role, camera, dimensions, stock/upgrade state, palette, material hierarchy, approved references and excluded features. Generate a small set of concepts, select one, then derive supporting views. Maintain a reference index so later tasks reuse the selected design rather than inventing another car or room.

Image outputs can serve as mood boards, paint explorations, decals after cleanup and reference sheets. They are not reliable orthographic measurements, mesh topology, UVs, seamless PBR sets or a consistent multi-view reconstruction by default. Generated UI screenshots are layout references; generate actual text, icons and widgets in vector/code/UMG. Do not make a functional menu by placing a generated screenshot behind invisible buttons.

Reusable request: Create a reference image for [ASSET_ID] in Last Signal. Follow the approved [REFERENCE_IDS], exact design brief and palette. Show [VIEW], [STOCK_OR_UPGRADE_STATE], and the complete silhouette without cropping. Keep surfaces physically plausible and readable; use restrained rust and wear. No labels, logos, invented text, extra wheels, impossible joints or unrelated objects. This is a visual reference, not a production mesh.

Blender to Unreal pipeline

1. Freeze the approved reference and dimensions. Build a scale blockout and export it before detailing.
2. Choose one tested axis/unit preset: Blender scene units in metres, applied transforms, export mapped to Unreal +X forward/+Z up. Validate with a 1 m cube and wheel axle test; do not assume a vehicle rig follows a static-mesh export preset.
3. Build clean topology, UVs and material IDs. Separate movable or replaceable components. Generate simple collision independently of visual detail.
4. Create the skeleton/skin where required, check normals and weighted transforms, then bake or author textures with a consistent channel convention. Document packed textures explicitly, for example ORM = occlusion/roughness/metallic.
5. Export with a pinned preset; import through a repeatable Unreal script. Reimport must preserve approved materials and references.
6. Inspect scale, pivots, wheel motion, collisions, LOD behaviour, lighting and damage states in the target map. Produce an asset report with measured issues.

Claude and Codex can both write Blender Python. Assign Codex the asset pipeline and visual review to reduce ownership conflicts, not because Claude is technically unable to do it. Use batch scripts for repeatable geometry and exports [S11]. Complex hero vehicles, clothing and faces still benefit from skilled manual art work or licensed base assets.

Characters and animation

Eli Vale, the playable lead, is an ordinary adult mechanic in a worn canvas jacket, layered shirt, practical trousers and work boots. Use arms/hands and a shadow/body representation first; a full cinematic head is unnecessary for driving and first-person service. Mara Vale, an adult emergency logistics coordinator, has a repaired blue coat and recognisable scarf; her identity comes through decisions and objects before her face appears.

Use MetaHuman Creator for hero human baselines if chosen. Assemble gameplay characters through the UE Optimized pipeline and verify LOD, hair and material costs [S08]. Start with hair cards or a measured economical hair setup; reserve cinematic settings for tightly controlled shots. Create original clothed characters; likeness and voice assets require permission from the relevant people.

Animation minimum: on-foot walk/idle, carrying a wheel, focus/reach, jack action, fastener action, wheel lift/seat, vehicle entry/exit transition, three enemy locomotion families, attach/hold/scrape-off, hit reactions. Use snap-assisted interactions and short transitions first. Avoid full-body first-person collision and complex finger simulation before the loop is fun. Do not use full MetaHumans for the road horde.

HUD layout and typography

At 1920 x 1080 reference resolution: keep essential information inside a 5% safe area. Bottom left: 260 x 120 vehicle condition cluster with body, wheel and fuel icons. Bottom right: 220 x 150 speed/boost display, with speed the largest number. Top centre: compact next-refuge name and remaining distance; no permanent paragraph of quest text. Clinger warnings appear on the affected screen side and include an icon plus direction, with optional sound/haptics.

Use a clean licensed sans-serif for body text and one restrained condensed face for labels. Prototype at 20-24 px body, 28-36 px section headers and 48-64 px speed readout, then scale for 720p, 1080p, 1440p, 4K and living-room distance. Offer UI scale and subtitle size. Texture wear belongs on frame edges; never distress essential text. Keep text as localisable strings and allow at least 35% expansion.

Screen	Composition	Controller behaviour
Driving HUD	Corner status, quiet centre, directional threats	No cursor; pause always available
Workbench	Categories left, 3D preview centre, stats/cost right	D-pad list, shoulder categories, confirm/back
Part comparison	Current and proposed with explicit tradeoffs	Focus change previews, confirm commits
Route board	Secured route, next refuge, optional detours	Focus nodes; reachable and locked use labels
Decoration	World preview with compact action strip	Stick move, bumpers rotate, confirm/place, back/cancel
Journal	Clue list left, readable evidence right	Next/previous; audio replay and transcript
Cosmetics	Paint groups and live vehicle preview	Preview free; purchase opens platform flow

All components need normal, focused, pressed, disabled, loading, error and selected states. Disabled actions explain why. Destructive choices require a separate confirm; ordinary navigation does not. Restoring a closed modal returns focus to its initiating control. Never require a mouse drag for inventory, decoration or wheel fitting.

Default controller map

Action	Driving	On foot or workshop
Left stick	Steering	Move
Right stick	Limited camera orbit	Look
RT / LT	Accelerate / brake then reverse	Placement fine controls when explicitly shown
South face button	Handbrake	Interact or confirm
West face button	Boost	Context action or rotate mode
East face button	Cancel in UI	Back or cancel carry
North face button	Look back	Inspect
LB / RB	Optional quick look left/right	Tabs or placement rotation
D-pad	Lights and accessible shortcuts	Navigate or nudge
Menu / View	Pause / route summary	Pause / journal

Button names resolve to the connected device's glyphs. Remapping is supported; a keyboard preset remains available. Add hold-to-toggle alternatives, deadzone/sensitivity settings, separate camera inversion, reduced shake/head bob/motion blur, subtitles, contrast options and no-mash service controls.

Audio identity

Layer engine load, RPM, tyre surface and loose body rattle; upgrades should change a recognisable sound without increasing perceived loudness unfairly. Impacts need onset, body resonance and debris tails, with voice limits. Warm garage room tone, distant wind and intermittent radio noise create contrast. Music is sparse, original/licensed and avoids constant action scoring. The story uses radio repetition sparingly; mandatory cues have subtitles and visual alternatives.

Reusable UI prompt

Design [SCREEN_ID] for Last Signal using the listed layout, palette and state model. Produce a 1920 x 1080 reference and a component/state inventory. Keep essential text clean and editable, edges subtly worn, centre visibility high, and controller focus unmistakable. Show current and proposed stats with units, costs, locked reasons and back navigation. Do not add premium currency, neon cyberpunk styling, fake tiny text or mouse-only controls. Implement approved widgets in UMG/Common UI with data bindings; the image is only the reference.

Claude and Codex setup and workflow

Division of work

Claude Code owns runtime implementation, build integration, gameplay data, save migrations and tests. Codex owns concept briefs, Blender generation scripts, asset manifests, UI visual references and independent review. The human owner approves driving feel, silhouettes, story, licences and release decisions. This division reduces conflicting edits; it is not a benchmark claim that one model is inherently better at either task.

Use one active Unreal editor controller at a time. Independent branches may contain code or Blender source work, but two agents must not concurrently mutate the same map, Blueprint, material or running editor. Assign the actual file write scope in each task. A handoff is a committed artifact with evidence, not only a chat summary.

Local preflight

1. Install the chosen UE 5.8 patch through Epic and the matching supported C++ build toolchain from its versioned documentation. Record exact versions; do not install compiler versions from an old tutorial blindly.
2. Create a C++ project named LastSignal. Start from a minimal template or a verified vehicle template; the template is scaffolding, not the final architecture. Create a dedicated test map and confirm the editor opens without missing modules.
3. Pin a stable Blender release and export preset. Install Claude Code using its current official instructions [S12]. Authenticate through the normal account flow. Install the chosen local Codex

client separately; its editor access must be verified rather than assumed from this chat.

4. Initialise Git and Git LFS. Track `.uasset`, `.umap`, `.blend`, source textures and large audio with LFS; ignore DerivedDataCache, Intermediate, Saved, generated IDE files and local credentials. Verify LFS files are actual files after a fresh clone.
5. Copy this pack into `Docs/LastSignal/`. Copy `agent_templates/CLAUDE.md`, `AGENTS.md` and `PROJECT_STATE.md` into the project root. Copy the BUILD_COMMANDS.md template to `Docs/LastSignal/BUILD_COMMANDS.md`. Keep one shared project-rules file to avoid contradictory instruction copies.
6. Compile and package an empty Windows build. Launch it outside the editor and record commands, versions and logs in the M00 handoff before adding systems.

Connect Claude Code to UE 5.8

Epic documents an Experimental built-in Unreal MCP server [S02]. Check that the installed patch actually exposes these settings and tools.

1. In Edit > Plugins, enable Unreal MCP and All Toolsets, then restart.
2. In Editor Preferences > General > Model Context Protocol, enable Auto Start Server, or use the editor console command `ModelContextProtocol.StartServer`.
3. In the editor console, run `ModelContextProtocol.GenerateClientConfig ClaudeCode` to generate the project configuration. For Codex, inspect its existing config before generating the corresponding client configuration.
4. Start Claude Code in the project root after the editor is running. Use `/mcp` to inspect connection status. The documented default endpoint is `http://127.0.0.1:8000/mcp`.
5. Discover the available tool schemas. First list actors, then create a disposable cube in the test map, save/reopen and delete it. Verify each result before automating production assets. Send editor mutations serially.

The server is local and lacks a default authentication layer; do not expose it through a public port or a VPN listener. This setup is a documented starting point, not a claim of connection in this session. If the tools differ, record the mismatch and use the fallback below rather than inventing tool names.

Reliable fallback and command templates

Use local file edits, Unreal Build Tool/Automation Tool and editor scripts when MCP is absent or insufficient. Enable Python Editor Script Plugin and Editor Scripting Utilities for editor automation. Keep gameplay in C++/Blueprint. Editor scripts must be rerunnable: resolve by stable path, validate existing assets, create only missing items and report changes. Binary `.uasset` and `.umap` files must be edited through supported editor operations, never as raw text.

The following are Windows PowerShell templates, not tested commands on your machine. M00 replaces the example paths with actual verified paths and records exit codes. Close the editor for reflected C++ layout changes and full clean validation; Live Coding is an iteration aid, not release evidence.

```
$UE_ROOT = 'C:\Program Files\Epic Games\UE_5.8'
$PROJECT_FILE = 'D:\Projects\LastSignal\LastSignal.uproject'
& "$UE_ROOT\Engine\Build\BatchFiles\Build.bat" LastSignalEditor Win64 Development $PROJECT_FILE -WaitMutex
```

```
& "$UE_ROOT\Engine\Binaries\Win64\UnrealEditor-Cmd.exe" $PROJECT_FILE -run=pythonscript
-script="D:\Projects\LastSignal\Scripts\Unreal\validate_assets.py" -unattended
```

```
& "$UE_ROOT\Engine\Build\BatchFiles\RunUAT.bat" BuildCookRun "-project=$PROJECT_FILE" -noP4 -platform=Win64
-clientconfig=Development -build -cook -stage -pak -archive -archivedirectory="D:\Builds\LastSignal"
```

`validate_assets.py` is a script to implement during M00/M01, not a supplied working Unreal integration. Tool availability and commandlet flags must be confirmed against the installed patch. Interactive editor operations cannot all run inside a commandlet.

For Blender, start with a batch script test using a disposable cube [S11]:

```
& 'C:\Path\To\blender.exe' --background --python-exit-code 1 --python '
D:\Projects\LastSignal\Scripts\Blender\build_test_cube.py'
```

Use a fresh process for reproducible asset builds; save `.blend` source and export logs. Do not overwrite a manually refined production asset without version control and an explicit asset task. An optional Blender MCP bridge is not required for this pipeline.

Small context workflow

Keep root instructions short. Store exact build commands, contracts and decisions in referenced files; Claude project memory supports repository instruction files [S13]. Give each session the task ID, current handoff, relevant source paths and a concrete reproduction or acceptance scenario.

Ask for one system or one coherent increment. First inspect existing code, then name the files to change. Require compiled implementation, a minimal test map or reproducible test, and recorded results. When the work is accepted, commit and update `PROJECT_STATE.md` before moving on. Read only changed interfaces and the next task at the next session.

Limit expensive visual loops: approve silhouette, then scale/import, then surface detail, then final render. Do not repeatedly generate polished images while pivots and wheel locations are unresolved. Batch similar props with one material family. Cache approved references and asset manifests. Avoid pasting large logs; include the first causal error and its context, then keep full logs on disk.

Task prompt template

Read CLAUDE.md, the current PROJECT_STATE.md, task [ID] and only its listed references. Inspect the implementation and dependencies. State the smallest coherent change and file scope, then implement it. Reuse current interfaces; do not rewrite unrelated systems. Compile and run the task's checks. Report exact commands and observed results, changed files, limitations and remaining work. Update the handoff. Stop at this task boundary; do not start the next system or add speculative features.

Asset handoff template

Build asset [ID] from the approved brief and reference set using the pinned Blender pipeline. First validate a scale blockout in Unreal; then detail. Deliver editable source, export, textures, collision, sockets, material assignments, preview and measured validation. Do not modify gameplay code. If the concept conflicts with dimensions or rig constraints, preserve the contract and flag the visual adjustment.

Review prompt template

Review task [ID] against its acceptance criteria and the diff. Prioritise save corruption, duplicate rewards, input conflicts, bad asset references, streaming failures and unbounded costs. Reproduce suspicious behaviour before asserting it. Return concrete findings with file paths and severity; make no changes during this review. List untested areas separately from failures.

Definition of done

Code or assets exist in the repository; the owner can reproduce the result; the named checks have evidence; no unrelated edits slipped in; data and save contracts remain compatible or have a migration; and the handoff states exactly what works. A screenshot of an editor viewport alone does not prove a packaged game feature works.

Testing performance and release gates

Candidate hardware targets

Aim for 60 fps during driving on a midrange PC profile, with scalable visual settings. A candidate development baseline is a Ryzen 5 3600-class CPU, RTX 3060 12 GB-class GPU, 16 GB RAM and SSD at 1080p output. This is an unvalidated target, not a published minimum specification. Establish the actual minimum from packaged-build testing. Your RTX 5080 development PC is useful for authoring, but it cannot stand in for slower customer hardware.

Use Lumen and Nanite selectively, with TSR or another validated upscaler where appropriate. Epic's Lumen guide ties scalability to specific frame budgets and internal resolution; enabling every highest-quality setting does not guarantee high fidelity at 60 fps [S04]. A 1440p/4K output label does not mean native rendering at that resolution. Keep transparent options for resolution scale, upscaling and frame generation if added; measure latency without relying on generated frames.

Initial project budgets: GPU work below 14 ms in the representative stress route, game thread below 8 ms and render thread below 10 ms, leaving room within a 16.67 ms frame. CPU and GPU work overlap; do not add these numbers together as a single frame budget. Allocate a starting 4 ms GPU to GI/reflections and profile the remainder by pass. Adjust quality or content if a measured bottleneck disagrees with the plan.

Runtime limits to test

- Starting maximum: 24 active road enemies, four attached clingers, eight short-lived ragdolls and twelve significant dynamic debris pieces. Reduce expensive states first when pressure rises; logical encounter difficulty must remain consistent.
- Start with 8 live road chunks including the current one; variations in chunk size or sightline may require changes. No growth in resident actor count across repeated streaming cycles.
- Candidate memory goals: below 10 GB process working set and 8 GB VRAM on the baseline profile, while retaining safety margins. Track texture residency, shader/PSO compilation hitches and audio memory separately.
- Measure a warm 10-minute route and a fresh cold-cache route in packaged builds. Report median, p95 and p99 frame times and every hitch over 50 ms. Provisional driving gate: median at or below

16.67 ms, p95 below 20 ms, p99 below 33 ms, and no recurring streaming hitch over 50 ms. These allow occasional misses and are not a claim of locked 60 fps.

- Run a 60-minute soak and accelerated route-index tests. After garbage collection, no sustained upward memory trend greater than 5% from the steady baseline; investigate the source rather than hiding it by restarting maps.

What to automate

Unit-level checks: stat evaluation order and clamps; negative-price rejection; recipe deductions; ownership invariants; duplicate checkpoint commits; schema migration; missing content fallback; seed/hash stability; and route progress excluding reverse/circle farming.

Functional checks: one complete run; failure and restart; controller-only wheel change; cancellation at every service stage; garage placement exclusion; a clinger attaching/detaching across chunk cleanup; save/load after checkpoint arrival; and resume after an ordinary quit. Test repeated confirm, rapid menu opening, device disconnect and missing assets.

Seed validation: at least 1000 generated layouts across each biome, checking connector compatibility, clear width, branch connectivity, pickup reachability and guaranteed critical rewards. Automated geometry checks do not prove fair encounters; review representative footage and test the hardest combinations manually.

Save fault injection: interrupt each write stage, corrupt one slot, remove a cosmetic definition, migrate an old profile, simulate disk-full and replay a reward transaction. Expected result: newest valid profile or clear fallback, no duplicated grants and no loss of prior banked inventory.

Manual playtest script

Give testers a controller and no explanation beyond the first in-game tutorial. Observe whether they can launch, dodge, recognise fuel danger, remove a clinger, bank salvage, identify an affordable upgrade, replace a wheel and relaunch. Record confusion, unnecessary menu actions, time spent servicing and whether they choose another attempt. Do not prompt the desired answer.

Track locally, with no remote telemetry service required: run start/end, reason for failure, distance, earned/spent resources, part choices, service duration, checkpoint arrival and frame-time samples. If remote analytics is later added, specify consent/data minimisation and platform requirements before collecting player data.

Accessibility and presentation

Verify text at couch distance; test UI scale at 720p and ultrawide. Every key audio cue has a visual equivalent. Provide subtitle backgrounds, speaker labels, reduced flashing, separate music/effects/dialogue volume, toggle holds, reduced shake and camera bob, readable night contrast and gore reduction. Include an assist mode with more forgiving fuel and damage, and record the chosen mode for local scores without blocking the story.

Release checklist

Build from a fresh clone with LFS assets resolved; validate clean cook and all maps, dynamic asset references and shader precaches. Test clean install, update over an older build, save migration, offline launch, controller hotplug, suspend/resume where applicable, language expansion, display changes and absent/corrupt optional DLC. Keep crash logs and symbols organised by build ID.

PC completion does not establish console readiness. Platform access, devkit testing, required SDKs, entitlement handling, storage rules, user switching, suspend/resume, certification and release scheduling need a separate platform-specific plan. Decide actual consoles before advertising support. No public documentation or AI connection substitutes for approved platform access.

Risk register

Risk	Early evidence	Response
Driving is not fun	Players quit after one run	Stop expansion; tune M03 and encounters
Manual repairs become chores	Service time dominates play	Improve Quick Fit and reduce required rituals
AI assets lack consistency	Wheelbase/materials drift	Lock references and technical manifests
Streaming stutters	Repeated p99 spikes	Prebake variants, shorten views, preload earlier
Scope overwhelms one developer	Tasks expand without gates	Reduce cars/regions before dropping quality
Experimental MCP breaks	Missing tools or changed schema	Use scripts and pin engine patch
Story feels manipulative	Twist invalidates player evidence	Keep objective clues and earned resolution
Economy forces grind	Several empty-value runs	Increase guarantees, reduce cost, remove debt
Saves lose progress	Interrupted-write failures	Fix transaction and A/B recovery before content

Production scope and monetisation

Product shape

Recommend a paid, complete single-player game with optional cosmetic DLC after a strong demo or slice proves demand. The first release should work offline once installed and must not require paid paints to enjoy customisation. Ship a useful selection of earned paint colours, decals, cabin objects and garage decorations.

The defining feature is the relationship between driving, a physical workshop and an intimate story. Spend production effort on vehicle feel, sound, visible upgrades and a few memorable places. Procedural volume does not replace authored variety. Build fewer excellent material families and reuse them deliberately.

R1 to R5 paint idea

Treat R1-R5 per paint as an affordability goal. Do not promise those exact standalone prices before checking the actual platform product type, regional price options and transaction requirements. Steam has pricing constraints, and an in-game purchase implementation has different obligations from selling a normal DLC product [S14, S15]. Platform pricing and launch-date rules need rechecking in the partner dashboard.

A simpler initial option is a small cosmetic DLC pack with several coordinated paints/decals, where the effective per-paint cost can be low without requiring many tiny purchases. An illustrative R19-R49 pack is a concept for research, not a recommended market price or guaranteed accepted tier. Validate willingness to pay and content cost after the slice. Do not build a bespoke wallet or backend just to sell a few colours.

All premium items are strictly cosmetic: no fuel, damage, capacity, crafting-speed or checkpoint advantage. Standard colours remain earnable. Avoid paid random packs, expiring ownership, countdown pressure and charging to recover a failed run. Let users preview paints under both garage and road lighting. Keep paid looks consistent with the somber world.

Entitlement design

Create an optional platform-entitlement adapter behind a stable interface. A paint definition references a cosmetic entitlement ID; ownership is resolved by the platform integration, not a boolean written into the ordinary save. Use developer mock ownership during production and keep store code out of the core loop.

For paid DLC, use the platform's documented ownership and install checks. For actual in-game transactions, use its required payment flow and authoritative grant confirmation [S15]. Never embed publisher secrets in a client. Handle cancellation, duplicate callbacks, pending state, offline ownership behaviour, refunds and unavailable products according to the chosen platform. Do not remove a player's build or corrupt their save if a cosmetic entitlement becomes unavailable; substitute a free appearance while preserving the chosen cosmetic ID for later restoration.

Art sourcing and licences

Maintain a provenance register for purchased assets, generated concepts, source meshes, fonts, audio, plugins and character likenesses. Record vendor, licence/version, receipt/location, allowed redistribution, modifications and the final asset IDs. Do not redistribute source packs inside the game's mod tools or documentation unless permitted. Check current engine, marketplace, MetaHuman and store terms when choosing assets and before release; this pack is a production checklist, not a legal determination.

A sensible cost strategy is licensed background materials/foliage, original hero vehicle shapes and story props, reusable procedural kits, and targeted specialist help for rigging, animation or audio where quality stalls. Avoid committing to paid asset packs until scale, art direction and export compatibility are verified in a small test.

Capacity and milestone control

Maintain four queues: current implementation, validated next tasks, asset backlog and deferred ideas. Limit work in progress to one runtime task plus one isolated art task. Every new feature names the existing feature or time allowance it displaces. Do not expand the campaign because procedural road generation becomes easy.

Before M15, commit only to the slice. After M15, reassess development hours, actual asset throughput, performance and player response. Before M21, decide whether cosmetics justify support and platform integration. Before M22, decide whether console ports have access, resources and a viable schedule. Gates are production decisions, not requests for the coding agent to seek permission after every reversible edit.

Open choices that can wait

Working title and final setting; exact target consoles; release price; how much art will be purchased versus authored; full voice acting versus text/radio fragments; and final scope if the slice overruns. Current assumptions let implementation begin without these decisions. Revisit setting before final environment art, voice before narrative recording, and business model before store integration.

Narrative bible and spoilers

Dramatic premise

Eli Vale follows his wife Mara along the Greywater evacuation route after their convoy separated during a collapse at a road barrier. He believes her voice is still coming through the relay network. Mara coordinated transport and medical supplies; her trail contains decisions she made for other people, not only clues left for her husband.

The question begins as Where is she and becomes When did this happen. The final answer is sad but concrete: Mara died helping evacuate people at North Relay before Eli's present journey began. Several messages he hears are automated rebroadcasts. His stress and guilt affect interpretation, while the physical evidence remains consistent. Do not imply that mental illness makes him violent or that every event was imaginary.

This ending is a proposed creative default, clearly separated from technical contracts. It can be changed before final clue recording, provided the timeline and evidence are rewritten together.

Objective timeline

Day 0: the convoy splits; Mara continues north with a relief group. Eli is injured and loses reliable time references. Day 1: Mara passes Mile Zero and marks a map. Day 3: she reaches Redline Depot and leaves a dispatch note. Day 5: she rests at Stillwater House and repairs her coat. Day 6: she reaches North Relay and helps route an evacuation. Day 7: she dies during a rescue, after recording a personal message. Day 10: Eli resumes the route. Fixed-date records expose the delay; the UI never falsely labels an old recording as live.

First three regions

Region 1, km 0-12: practical hope. Mile Zero contains the marked map and a familiar cup. At km 4, the road log confirms she travelled through. At km 8, an object suggests a deliberate detour. A radio sign-off repeats with the same small click, initially easy to dismiss. The first garage tutorial is motivated by preparing the car she helped maintain.

Region 2, km 12-30: contradictory chronology. Redline's handwritten dispatch book shows Day 3, while a damaged printed receipt uses an earlier date. Investigation explains that the printer clock was wrong rather than relying on a supernatural trick. At km 18, a witness note mentions her blue coat and another traveller. At km 24, the route stamp proves her stop occurred days before Eli expected. Replayed radio phrasing becomes recognisable. The player should begin questioning Eli's assumptions, not the game's honesty.

Region 3, km 30-48: intimacy and acceptance. Stillwater looks inhabited because an absent caretaker prepared it for future arrivals. Mara's repaired coat and recording instructions reveal that messages were meant to repeat. At km 36, a relief manifest names people she helped. At km 42, the final personal recording is recovered but its damaged last segment cannot play until the relay is reached. At km 48, an objective log and a named survivor's written testimony confirm her death. Her complete message asks Eli to continue living, without magically knowing his exact future route.

Finale and continuation

The final driving leg culminates in reaching the relay through a visible evacuation choke, not an unrelated boss arena. Inside a small safe room, the player restores power and hears the complete recording, with subtitles and a skippable transcript. No timed combat interrupts it.

Choose whether to leave the repaired radio at Stillwater or keep it in the car. This affects one home prop and a quiet epilogue line, not a paid item or contradictory ending. The campaign ends clearly. Endless mode is a post-story continuation of Eli maintaining supply routes; the search narrative does not keep pretending Mara is alive.

Clue implementation and restraint

Clue ID	Location	Observable fact	Interpretation
C01_MAP	G01	Mara marked a northward route	She intended to reach the relay
C02_LOG	CP04	A dated entry matches the convoy	Eli assumes she is close
C03_TOKEN	CP08	Her familiar luggage ribbon	The detour was deliberate
C04_LEDGER	G02	Two records disagree	A clock error, later explained
C05_NOTE	CP18	Someone saw her helping another traveller	She was not travelling alone
C06_STAMP	CP24	A consistent timestamp is older than expected	Eli's chronology is wrong
C07_COAT	G03	Repair and recording instructions	The voice may be a rebroadcast
C08_MANIFEST	CP36	Names of people she helped	Her choices had lasting value
C09_RECORDING	CP42	A damaged personal message	The final answer is at the relay
C10_TESTIMONY	CP48	Death confirmed by independent records	Search becomes remembrance

Clues are guaranteed and journalled once; optional echoes can vary. Use no more than one overtly uncanny beat per major leg: a repeated radio click, an object remembered in the wrong place, or a reflection that resolves on a second look. Such moments never alter collision, hide essential hazards or change already recorded evidence. Avoid jump-scare spam and the reveal that the whole journey never happened.

Writing and delivery style

Short practical notes with personal details beat exposition. Mara's voice is competent, tired and caring, without saintly perfection. Eli speaks sparingly; breathing, hesitation and the car's familiar sounds carry much of the emotion. Write evidence text before recording dialogue. Maintain a clue timeline spreadsheet or JSON and review every revision for date consistency and accidental spoilers in filenames exposed to players.

Sources assumptions and decision log

Evidence status

Technical sources were checked on 29 October 2026. Documentation is version-sensitive. Recheck the installed engine patch, plugin tool schemas and store dashboard before execution. The sources establish tool capabilities and constraints; the game's mechanics, budgets, prices, content and estimates are original proposals requiring validation.

S01. Epic Games, Unreal Engine 5.8 release announcement. Confirms the release and distinguishes Experimental features.

https://www.unrealengine.com/news/unreal-engine-5-8-is-now-available

S02. Epic Games, Unreal MCP in Unreal Editor, 5.8. Connection setup, local endpoint, toolsets and Experimental status.

https://dev.epicgames.com/documentation/unreal-engine/unreal-mcp-in-unreal-editor?application_version=5.8

S03. Epic Games, Using PCG Generation Modes. Runtime generation and cleanup are available; the game's bounded road scheduler remains a separate design.

https://dev.epicgames.com/documentation/en-us/unreal-engine/using-pcg-generation-modes-in-unreal-engine

S04. Epic Games, Lumen Performance Guide. Quality, resolution and frame-budget tradeoffs.

https://dev.epicgames.com/documentation/unreal-engine/lumen-performance-guide-for-unreal-engine?lang=en-US

S05. Epic Games, Scripting the Unreal Editor Using Python. Editor automation rather than packaged gameplay.

https://dev.epicgames.com/documentation/en-us/unreal-engine/scripting-the-unreal-editor-using-python

S06. Epic Games, Chaos Vehicles. Vehicle simulation starting point.

https://dev.epicgames.com/documentation/unreal-engine/chaos-vehicles

S07. Epic Games, Nanite Virtualized Geometry. Supported content and practical performance constraints; consult the installed version for feature maturity.

https://dev.epicgames.com/documentation/en-us/unreal-engine/nanite-virtualized-geometry-in-unreal-engine

S08. Epic Games, MetaHuman Assembly. Gameplay-oriented assembly and optimisation options.

https://dev.epicgames.com/documentation/metahuman/assembly?lang=en-US

S09. Epic Games, Enhanced Input. Contextual actions and remapping.

https://dev.epicgames.com/documentation/en-us/unreal-engine/enhanced-input-in-unreal-engine

S10. Epic Games, Common UI. Layered UI, controller focus and glyph support.

https://dev.epicgames.com/documentation/en-us/unreal-engine/common-ui-plugin-for-advanced-user-interfaces-in-unreal-engine

S11. Blender Foundation, Command Line Arguments, versioned 3.6 reference. Establishes the background/Python workflow; validate flags and API against the pinned installed Blender release. The latest manual endpoint could not be retrieved in this research session.

https://docs.blender.org/manual/en/3.6/advanced/command_line/arguments.html

S12. Anthropic, Claude Code MCP documentation. Client connection and tool discovery; follow the official site navigation for current installation instructions.

https://code.claude.com/docs/en/mcp

S13. Anthropic, How Claude remembers your project. Project instruction files and scoped context.

https://code.claude.com/docs/en/memory

S14. Valve, Steamworks Pricing. Product pricing and regional/minimum constraints; exact product prices require partner validation.

https://partner.steamgames.com/doc/store/pricing

S15. Valve, Microtransactions. In-game purchase requirements, distinct from a normal DLC product.

https://partner.steamgames.com/doc/features/microtransactions

Decision log

ID	Decision	Reason	Revisit
D01	PC first, consoles later	Establish performance and fund platform work	After slice
D02	Single-player offline core	Keep scope and save ownership simple	Separate sequel/expansion
D03	Chase driving and first-person garages	Strong car readability and tactile service	M03/M10 feedback
D04	Three cars and three garages	Finite content budget	M15 scope review
D05	Authored chunks with procedural dressing	Fair driving and reliable narrative	M16 profiling
D06	Physical service plus Quick Fit	Preserve ritual without forced repetition	M15 playtests
D07	Cosmetics only, store deferred	Protect progression and production time	M21 business gate
D08	Stable part IDs and transaction ledger	Protect saves and rewards	Schema changes
D09	Optimised hero humans, simpler horde	Spend character cost where visible	Character profiling
D10	Authored ending, optional eerie details	Give the search a coherent resolution	Before recording

Remaining validation

No UE build, Blender export, MCP connection, gameplay benchmark or console test was performed in this planning session. All commands in the setup guide are labelled templates. The balance seeds, technical type names, dimensions, asset counts and estimates are specifications to implement and test. The PDF and pack structure were checked as documents; this does not certify the planned game systems.